Agentic Pilot: An Evidence-Driven Local Autonomous AI Agent Framework for Privacy-Preserving Intelligent Task Automation

***Abstract*—Browser agents can report a task as finished when it is not, and many send every observation to a hosted model. Agentic Pilot is an open-source browser agent whose models run locally through Ollama and which may report completion only when a rule-based predicate over the observed page holds; the inputs of each decision are stored as evidence. We formalize this evidence-gated completion, separating action dispatch from task completion, and measure it with false completion and false rejection rates. In a component study inside the agent's live verification node, the gate withheld completion in 20 of 23 unsatisfied design states (95% interval 0.68–0.95) that a dispatch-only rule accepts, and rejected 1 of 15 satisfied ones. On 42 held-out states frozen before testing it withheld 9 of 22 (0.23–0.61; exact McNemar $p = 0.004$) and none of the 5 unsatisfied extraction states, because its answer check is triggered by request wording and accepts any answer that repeats a goal term. Removing any single conjunct changed at most three held-out decisions. A decision costs a median of 27 ms and no model call. We enforce that model traffic stays on the machine; in 70 agent episodes the agent process opened no non-loopback connection. We state five testable hypotheses on verification, traceability, memory and orchestration, and release an end-to-end harness with self-hosted sites, 40 tasks, baselines and an independent oracle that agreed with all 114 scripted trajectories; no end-to-end run with a language model is reported.**

***Index Terms*—LLM agents, web agents, local inference, privacy-preserving computing, completion verification, task automation, human-in-the-loop**

I. Introduction

Language-model agents now operate software for people: they open pages, fill in forms and read results [1]–[3]. Many send each observation, often a screenshot of a logged-in page, to a hosted multimodal model [2], [4], [5], although such screens carry names, messages and account data and personal information has been shown to leak from language models [6]. Independently of where the model runs, an agent can stop too early. We call this *premature completion*: a terminal "done" while the environment does not satisfy the goal, for example after opening the search engine but before issuing the query. Each action succeeded; the task did not.

a) Research gaps: We address three gaps, each stated so that it can be tested.

G1 *In-loop completion checks are rarely measured.* Benchmarks check the final state after the episode [1], [3], and judges assess saved trajectories [2], [7]; when the agent itself decides, it is usually its own model that does so [5], [8], and intrinsic self-correction is unreliable [9]. Whether a rule over observed state inside the loop withholds premature completions, and at what false-rejection cost, has not been reported with intervals and held-out data. Test: Sections VII-A and VIII-A.

G2 *The contribution of each kind of evidence is unknown.* If every conjunct of such a rule contributes, removing it should raise false completions. Test: the ablation of Section VIII-B.

G3 *Locality claims of local agents are rarely checked.* Running models locally is claimed to keep data on the machine, but whether an agent's process contacts other hosts is seldom measured. Test: the egress test of Section VIII-D.

b) Scope and terms: Agentic Pilot acts only in a Chromium browser. It is *autonomous* in that, given a request, it plans and executes browser actions without step-by-step instructions; the user approves tasks classified as high-risk (posting, sending, purchasing, deleting, transferring) and solves CAPTCHAs, and no end-to-end task with a language model is evaluated in this paper. It is *privacy-preserving* in that model inference is restricted to the user's machine and its records are stored there (Section X). Desktop, file and process control is a design (Appendix A).

c) Scientific contributions:

1) A formulation of evidence-gated completion that separates action dispatch from task completion, with false completion and false rejection rates (Section III).
2) Evidence on G1 and G2: the gate's error rates on 92 constructed states in three sets, one frozen before testing, inside the live verification node, with intervals, paired tests and a per-conjunct ablation. The main finding is negative for generalization: rule fixes made on design states transfer only partially (Section VIII).

d) Engineering contributions:

3) An open-source local agent with the gate inside its control loop (Algorithm 1, Section IV), a code-level component audit (Table II) and the fixes it led to (Section V).
4) Enforcement and a test of local-only model traffic, addressing G3 (Sections V and VIII-D).
5) An end-to-end harness: self-hosted sites with fictional content, 40 tasks with a dev/test split, an oracle validated on scripted trajectories, and baseline and ablation configurations, with five research hypotheses and the design that tests them (Sections VI and VII).

We do not claim that the agent completes tasks; the end-to-end evaluation is designed and implemented but not run (Section VIII-E). Appendix C illustrates its reporting with hypothetical values that are not results.

II. Related Work

a) Reasoning-and-acting agents: ReAct interleaves reasoning with tool actions [8], building on chain-of-thought prompting [10]; Toolformer learns when to call an API [11]. Reflexion and RCI let the model critique and retry its own attempts [12], [13], and Voyager accumulates skills checked by a model [14]. In all of these a model decides that a task is finished, and intrinsic self-correction without external feedback does not reliably help [9].

b) Web and computer-use agents and benchmarks: Mind2Web pairs a ranker over Document Object Model (DOM) elements with an LLM [15]; WebVoyager drives live sites from screenshots [2]; SeeAct finds grounding to be the bottleneck [4], [16]. WebArena, VisualWebArena and WorkArena evaluate on self-hosted or remote-hosted applications with functional checks [1], [17], [18]; OSWorld and Windows Agent Arena do so in real operating systems [3], [19]. UFO, Agent S and AppAgent control desktop and mobile applications [5], [20], [21], and CogAgent, SeeClick, UI-TARS and OmniParser ground actions in screenshots [22]–[25]. These works say little about when the agent itself may conclude that a task is done.

c) Completion checking: Benchmark checkers run after the episode. WebVoyager evaluates trajectories with a multimodal judge [2], Pan et al. use model-based evaluators to judge and refine agents [7], and Sumyk and Kosovan train a completion judge for macOS applications [26]. We study a rule over observed state *inside* the loop, record its inputs, and measure its errors (G1, G2); a model judge is complementary.

d) Memory, security and local inference: Retrieval-augmented generation [27], [28], Generative Agents and MemGPT [29], [30] manage agent memory. Indirect prompt injection hides instructions in content the agent reads [31]; defenses are insufficient in systematic evaluations [32], InjecAgent, AgentDojo, ToolEmu and OS-Harm measure unsafe behavior [33]–[36], and CaMeL separates trusted control flow from untrusted data [37]. Open-weight models such as Qwen2.5 and Qwen2-VL run on consumer hardware [38]–[40]; surveys list trustworthiness among open problems of LLM agents [41], [42]. Table I compares representative systems.

TABLE I

Comparison with representative systems, from what each paper reports. Kind: A agent, B benchmark, M model. Environment: W live websites, Wr self-hosted or cached web pages, Ws simulated web, tool or text environment, D desktop OS, Mo mobile. Completion: who decides that a task is done—the agent's model (model), a checker of the benchmark after the episode (benchmark), a model judge after the episode (judge), or a rule over observed state inside the loop (in-loop rule); n/a for systems that do not end tasks. Evid.: persists state artifacts of each step. Mem.: memory across steps or episodes. Open: open-weight models used or supported. Y yes, P partial, N no, n.r. not reported. All listed systems released code.

System	Kind	Env.	Completion	Evid.	Mem.	Open
ReAct [8]	A	Ws	model	N	N	N
Reflexion [12]	A	Ws	model[a]	N	Y	n.r.
WebArena [1]	B	Wr	benchmark	n.r.	N	n.r.
Mind2Web [15]	B	Wr	n/a	P	N	Y
WebVoyager [2]	A	W	model; judge[b]	Y	N	N
SeeAct [4]	A	W, Wr	model	n.r.	N	P
OSWorld [3]	B	D	benchmark	n.r.	N	Y
UFO [5]	A	D	model	n.r.	P	N
AppAgent [21]	A	Mo	model	n.r.	Y	N
CogAgent [22]	M	Wr, Mo	n/a	N	N	Y
MemGPT [30]	A	n/a	n/a	N	Y	n.r.
AgentDojo [34]	B	Ws	benchmark	n.r.	N	Y
Agentic Pilot (ours)	A	W[c]	in-loop rule	Y	P[d]	Y

[a] An evaluator gives feedback between trials. [b] Used for evaluation. [c] Implemented backend; a desktop backend is designed (Appendix A) and not integrated; our end-to-end harness uses self-hosted sites (Wr), not yet run with a model. [d] Implemented and retrieved; its effect is not measured.

III. Problem Formulation

a) Dispatch and completion: The agent observes an environment state s_t through an observation $\Omega(s_t)$ (for the browser: URL, title, element list and screenshot). A planner maps a goal g, a plan $P = (g_1, \ldots, g_n)$ and the observation to an action a_t. Executing it returns a *dispatch flag* $\alpha_t \in \{0, 1\}$, which says only that the call returned without error. The agent may report the task complete only when a completion predicate holds on the state observed *after* the action:

$$\Phi(g, s_{t+1}, P, \sigma) = \bigwedge_{c \in C(g,P)} c(s_{t+1}) \wedge \pi(P) \wedge \neg\beta(s_{t+1}, \sigma), \quad (1)$$

where $C(g, P)$ is the set of postconditions that the goal and plan imply, $\pi(P)$ states that no plan step is pending, and β is a blocking condition (a bot-check page, an unapproved high-risk action) with σ the approval state. Dispatch does not imply completion, $\alpha_t = 1 \not\Rightarrow \Phi = 1$. Every evaluation of Φ is stored with the observed values it used. The same form applies to desktop, file and process state, whose postconditions are typed and exact (Appendix A); only the browser instance is implemented.

b) The implemented browser predicate: $C(g, P)$ is derived from the wording of the request. Φ is the conjunction of: no browser-error URL, error or "not found" title, or redirect to a sign-in page the request did not name (E); not still on a search engine's home page when a search, an extraction or a multi-step plan was requested (H); a results page carrying the query quoted in g (Q); the literal text that g asks to be entered present in an input, exactly if g says "exactly" (T); for extraction requests, an answer or structured extraction that mentions a content word of g (X); and, for multi-step plans, no pending step unless a structured extraction exists (R). Each conjunct is instantiated only when its trigger words occur, so for a request that triggers none, Φ reduces to $\neg E$. A pre-check implements β for bot-check pages and halts the task as Blocked. A plan step counts as done when an action

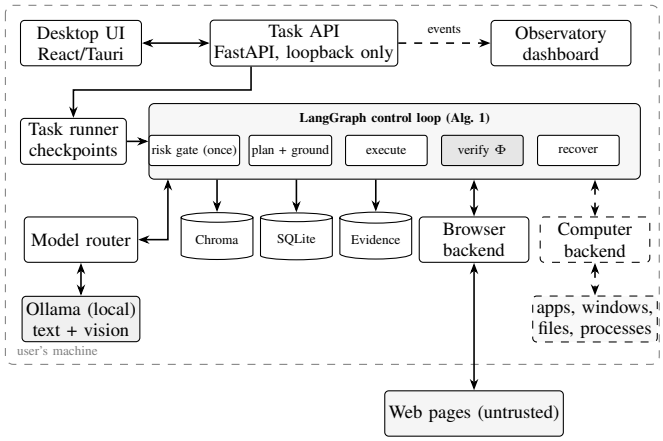


Fig. 1. Components of Agentic Pilot. Solid: implemented; dashed: proposed. Everything inside the boundary runs on the user's machine; the model endpoint must be a loopback address unless the user overrides it. The browser backend is the only implemented execution environment; the computer backend and the local resources it would act on are a design (Appendix A). The Observatory also reads the task database and evidence files.

other than `complete` dispatches during it. Φ has no tunable threshold: its operating point is fixed by the rules, so we characterize the trade-off between false completions and false rejections by removing conjuncts (Section VIII-B) rather than by a curve.

c) Metrics: Let $\mathcal{T}$ be a set of episodes or end states, $\mathcal{T}_c \subseteq \mathcal{T}$ those reported complete, and $\Phi^\star$ an oracle written independently of Φ (Section VII). With $N_{\mathrm{FC}} = |\{\tau \in \mathcal{T}_c : \Phi^\star(\tau) = 0\}|$ and $N_{\mathrm{FR}} = |\{\tau \in \mathcal{T} \setminus \mathcal{T}_c : \Phi^\star(\tau) = 1\}|$,

$$\mathrm{FCR} = \frac{N_{\mathrm{FC}}}{|\mathcal{T}_c|}, \quad \mathrm{FRR} = \frac{N_{\mathrm{FR}}}{|\{\tau \in \mathcal{T} : \Phi^\star(\tau) = 1\}|}. \tag{2}$$

Because a dispatch-only rule reports every state complete, we also report the share of unsatisfied states that the gate withholds, $1 - N_{\mathrm{FC}}/|\{\tau : \Phi^\star(\tau) = 0\}|$. FCR alone rewards an agent that never finishes, so the rates are always reported together, as counts with Wilson 95% intervals [43].

IV. Architecture

Fig. 1 shows the components. A desktop client submits a natural-language task to a FastAPI service bound to the loopback interface. The task runner executes the control loop, checkpoints its state after every node and cancels tasks that exceed a wall-clock limit (5 min by default). Task events go to SQLite and to a read-only monitoring process.

A. Control Loop

The loop is a compiled LangGraph state machine with eleven nodes; Algorithm 1 gives its behavior. A typed state record carries the intent, the plan and step index, the observation, the planned action, the action history, retry and model-call counters, routing decisions and the status. Only the verification step can end a task as COMPLETED, and only when Φ holds (Fig. 2).

Algorithm 1 Evidence-gated control loop (browser backend)

Require: goal g; budget of K=15 model calls
$I \leftarrow \textsc{ParseIntent}(g)$; $P \leftarrow \textsc{Decompose}(g)$ if multi-step
if I is high-risk and not approved **then return** WAITAPPROVAL
end if
$\textsc{Navigate}(I.site)$; on failure go to line 14
loop
  $o \leftarrow \textsc{Observe}$ ▷ sanitized DOM, screenshot
  if $\beta(o)$ **then return** BLOCKED ▷ hand off to user
  end if
  $a \leftarrow \textsc{Plan}(g, P, o, \textsc{Retrieve}(g))$ ▷ model call
  $\alpha \leftarrow \textsc{Execute}(a)$; save before/after evidence
  if $\alpha = 0$ **then**
    if fewer than 3 retries **then continue**
    end if
    st $\leftarrow \textsc{Recover}$(error)
    if *st* is exhausted or blocked **then return** FAILED
    end if
    continue ▷ retry, vision, ...
  end if
  if $a \neq$ `complete` **then** mark step done; **if** steps remain **continue**
  end if
  if $a =$ `complete` or last step or text typed **then**
    $(v, \mathrm{obs}) \leftarrow \Phi(g, s, P)$; save as evidence
    if v **then** store memory; **return** COMPLETED
    end if
  end if
  if model calls $\geq K$ **then return** FAILED
  end if
end loop

a) Planning: A small model parses the request into an intent (action, site, target, content, risk level). Requests with a connective or more than eight words are decomposed into steps with an action type and expected outcome. Each iteration, the planner sees the goal, the plan, the current observation and retrieved memory and returns one action. Deterministic rules then adjust it: exact-text requests are grounded to the best visible text field, repeated navigation becomes a failure, and plan steps that name clicking or extracting map to those actions.

b) Recovery, routing and memory: A failed action is retried through a fresh observation up to three times; later failures are classified into seven types and assigned a strategy (retry, alternative selector, vision fallback, replan), with at most six attempts. Only the vision-fallback strategy changes behavior, by routing the next step to the vision model; the other strategies repeat ordinary planning. A deterministic router assigns each model call a role and the first installed candidate among local Qwen2.5, Qwen3.5 and DeepSeek-R1 models (1.5B–7B parameters), or `qwen3-vl:2b` and `moondream` for vision. At termination, outcomes are written to a local Chroma collection and retrieved into later planning prompts.

B. Browser Backend

A pool of Playwright contexts over one Chromium instance gives each task its own page. Each observation is a DOM pass over interactable elements, recording tag, role, label, up to 120 characters of text, type, visibility, bounding box and selectors; elements are ranked, deduplicated and capped at 50, and their text is passed through a prompt-injection sanitizer. Actions are navigate, click, type, key press, scroll and extract;

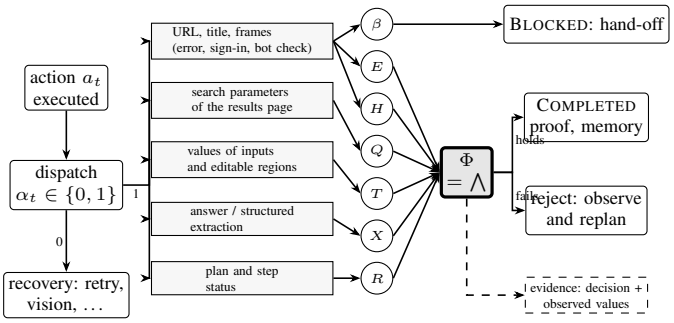


Fig. 2. Evidence-gated verification after an action (Algorithm 1, lines 7 and 22). A failed dispatch goes to recovery and never to Φ. A successful dispatch makes the observed values available to the conjuncts of the browser predicate, each triggered by the wording of the goal (Section III); the bot-check condition β is tested first and hands the task to the user. Every decision is stored with the values it used.

typed text is read back. The vision model is used only when the planner returns `need_help` or recovery requests it: it receives a screenshot and returns a point, which is clicked without an individual check. Per task, the evidence directory holds before/after screenshots of each action, the latest DOM manifest, the verification and recovery records and the node trace.

V. IMPLEMENTATION

The backend is Python (about 11,000 lines) on FastAPI, LangGraph, Playwright and Pydantic, with SQLite and a persistent local Chroma store; model calls use the Ollama HTTP API with JSON output and a fixable temperature and seed. The client is React in a Tauri shell. The code is public under the MIT license at https://github.com/sudeshsudhii/Agentic-pilot-local; this paper describes commit `bf0a8a5`.

a) Component audit: Agent documentation tends to run ahead of the code, so we traced each component to the code path that executes it and to the tests and experiments that exercise it (Table II). Claims in this paper are restricted to rows with evidence in the last column; the other rows are implemented but unevaluated, or designs.

b) Fixes: The audit and the gate study led to fixes. At commit `2361d50`, the extraction handler formatted every answer as facts about one university campus and substituted fixed text when a page had too few matching lines; it now keeps only page lines that share terms with the goal. The verification node accepted the reasoning of any action as an answer and let a `complete` action mark a pending step done; both are fixed, and the Q, sign-in and "not found" checks, exact matching and goal-term overlap were added. Bot-check detection no longer fires on titles that contain "robot" or on hidden frames. The vision-fallback strategy, memory retrieval and the verification switch were inert and now take effect; routing keywords match whole words. The repository's earlier "certification" figures counted a task as passed when it ended as completed and a screenshot existed, and we use none of them.

TABLE II
COMPONENT STATUS, TRACED THROUGH THE CODE. IMPL.: CODE EXISTS (P PROTOTYPE OR PARTIAL). INTEG.: ON THE AGENT'S EXECUTION PATH. TEST: Y BEHAVIOR TEST; M WITH A MOCKED MODEL; S SCRIPTED STAND-IN MODEL; I INTERFACE ONLY; W NEEDS LIVE WEB. EVAL.: EVIDENCE IN THIS PAPER—C COMPONENT STUDY, O ORACLE VALIDATION, E EGRESS TEST, R ROUTING RESOLUTION; – NONE. CLAIMS IN THIS PAPER ARE LIMITED TO ROWS WITH AN ENTRY UNDER EVAL.

Component	Impl.	Integ.	Test	Eval.
Completion predicate Φ, bot-check halt	Y	Y	Y	C
Browser control (Playwright)	Y	Y	Y	C
Local-only model endpoint, no telemetry	Y	Y	Y	E
End-to-end harness and oracle $\Phi^\star$	Y	–	S	O
Model routing	Y	Y	Y	R
LLM planner in the full loop	Y	Y	M, S	–
DOM grounding; vision fallback	Y	Y	Y; M	–
Per-action evidence	Y	Y	Y	–
Recovery: retry, vision fallback	Y	Y	Y	–
Recovery: alt. selector, replan	P	P	Y	–
Memory and RAG retrieval	Y	Y	Y	–
Self-verification baseline mode	Y	Y	S	–
Task-level human approval	Y	Y	W	–
Desktop input, files, processes	P	N	I	–
Accessibility tree, action gate, tiers	N	N	N	–

c) Locality enforcement: The model client now refuses any endpoint whose host is not a loopback name or address, before connecting, unless the user sets an explicit override; LAN addresses count as remote. Chroma stores are created with telemetry disabled, and the privacy audit log classifies destinations by parsed host instead of by substring. For experiments we added switches that replace Φ by no check or by a self-check of the planner model, remove single conjuncts, or pin one recovery strategy, and the gateway records the token counts that Ollama reports.

VI. RESEARCH HYPOTHESES

The following hypotheses state what the full agent is expected to show. *None has been tested end to end*; Section VII-C gives the design for testing them, and Appendix C shows, with hypothetical numbers only, how the results would be reported. Table III summarizes the variables.

a) H1, completion verification: Because a successful dispatch does not imply a completed task (Eq. (1)), and the gate withheld most premature completions on constructed states, evidence-gated completion is expected to reduce the false completion rate of the full agent relative to accepting every completion claim, without a comparable loss in task completion.

b) H2, verification reliability: A planner model that judges its own completion shares the errors of its own plan [9], whereas Φ is deterministic given the observation. Rule-based verification is therefore expected to give a higher verification accuracy and a lower variance across trials than self-

TABLE III
RESEARCH HYPOTHESES (UNTESTED). IV: INDEPENDENT VARIABLE; DV: DEPENDENT VARIABLES (SECTION VII-C). COMPARISON: CONFIGURATIONS OF TABLE VII AGAINST `FULL`; †NOT YET IMPLEMENTED.

	IV	DV	Expected	Comparison
H1	completion check on/off	FCR, TCR, FRR	FCR ↓, TCR ≈	no verification
H2	rule check vs model self-check	VA, FCR, spread over trials, latency	VA ↑, latency ↓	self-verification
H3	execution records on/off	ETR, audit time	ETR ↑	no evidence
H4	memory and RAG on/off	TCR on trials 2–3, MCS, actions	TCR ↑, MCS ↑	no memory
H5	state machine vs unstructured loop	AER, recovery rate	AER ↑	unstructured loop†

verification by the same model, at a lower latency. The held-out misses (Section VIII-A) make the accuracy part uncertain.

c) H3, execution traceability: The evidence directory stores before/after screenshots, the DOM manifest, verification and recovery records and per-action execution records. With this evidence, an auditor is expected to decide whether a reported completion was correct for more episodes, and faster, than from the event log alone. The configuration switch `enable_evidence` disables only the per-action execution records, so H3 as testable today compares full evidence with evidence lacking those records.

d) H4, persistent memory: Outcomes stored in Chroma and SQLite are retrieved into later planning prompts. When a task is repeated, memory is expected to raise success on later trials and the consistency of outcomes across trials, and to reduce actions per successful task. Memory may also mislead the planner when a stored outcome was wrong, so H4 also counts such cases.

e) H5, structured orchestration: The agent is a single LangGraph state machine with explicit nodes, retry limits, recovery and a model-call budget, not a set of cooperating agents. This structure is expected to raise execution reliability—the share of episodes ending in a defined terminal state without a crash or time-out—compared with an unstructured loop in which the model is called repeatedly with the page and history until it emits `complete`. *The unstructured baseline is not implemented*; testing H5 first requires building it with the same models and budget.

VII. EVALUATION METHODOLOGY

The experiments need no language model. The current-code results were produced on 2026-10-04 on a Linux container with a 4-core Intel Xeon at 2.1 GHz, 15 GiB RAM and no GPU, with Python 3.11.15, Playwright 1.56.0, headless Chromium 141 and LangGraph 1.2.12; the decisions of the code before the fixes come from an earlier run on a similar container and are deterministic. Scripts, seeds and raw outputs are under `artifacts/` and `benchmarks/e2e/` in the repository.

A. Gate Study

a) States and splits: A state is a goal, a URL, an HTML document served through Playwright request interception, the agent's answer and plan state where relevant, and an oracle label. Every state ends with a dispatched action, so a dispatch-only rule accepts all of them. The *design* set (38 states in six categories) was used to find the gate's defects and design the fixes. *Held-out 1* (12 states) was written after the fixes and not used to change them. *Held-out 2* (42 states, 22 unsatisfied) was written to a fixed procedure—varied user phrasings per category, including wordings the gate's trigger lists do not contain, with a balanced split—and committed before the gate was run on it; no gate code changed afterwards. Its SHA-256 is recorded in the repository.

b) Oracle: The label $\Phi^\star$ of a state states whether the goal is achieved by the page and answer shown, judged from the page's facts (for example, whether the answer contains the year in the article). It was written together with the state, before the state was judged by the gate, and it is computed by no code shared with Φ. Its limitation is that one author wrote all states and labels, so no inter-annotator agreement exists; a protocol for independent relabelling with Cohen's κ [44] is in the repository.

c) Modes and statistics: Each state is judged by the pre-check and Φ called directly, and by the real verification node (database, browser-pool and evidence I/O stubbed) with an empty or a generic one-sentence planner reasoning. For the ablation, each conjunct of Φ and the pre-check β is removed in turn by a configuration switch. Counts carry Wilson and Clopper–Pearson 95% intervals [43], [45]; comparisons on the same states use the exact McNemar test [46] on discordant pairs. Latency is the median of 20 repetitions after a warm-up.

B. End-to-End Harness

a) Environment and tasks: Live websites change and block automated traffic, and WebArena-style site images were not available offline, so we built five self-hosted sites served on the loopback interface with fictional content that a model cannot answer from memory: a search engine, an encyclopedia, a shop with search and cart, a town-office contact form and newsletter, and a notes application; a further path redirects to a sign-in wall. The 40 tasks (10 development, 30 test) cover search, navigation, text entry, extraction, form submission, multi-step tasks and 5 unsatisfiable requests, at four difficulty tiers by the number of actions a solution needs (1, 2–3, 4–6, more). The notes site appears only in the test split. Tasks needing an account, payment, a CAPTCHA, the live web, or an outcome that neither server state nor answer text can verify were excluded by design.

b) Oracle: $\Phi^\star$ is computed from signals that neither the agent nor Φ uses: server-side records of the sites (cart contents, form submissions, saved notes, field values autosaved by page scripts), the final URL, and the ground-truth facts in the site data matched against the agent's answer; unsatisfiable tasks are never achieved. It calls no model and shares no code with the

verifier. We measured its error rate on 114 scripted trajectories driven directly through Playwright without the agent: a correct solution for each task, an idle trajectory that only opens the start page, and a near miss for 34 tasks (wrong value, wrong item, extra submission, partial query).

c) Protocol: Table VII (Appendix B) lists the configurations: the full system, two baselines—(a) no completion check and (b) self-verification by the planner model—and one-factor ablations of each conjunct, memory, recovery and each recovery strategy. Each configuration runs in its own process with its own database and memory store, at temperature 0 with a fixed seed, three trials per task, and the same budget of 15 planner calls. Per episode the harness records the oracle verdict, the agent's status, browser actions, model calls and tokens, wall-clock time, each verification decision with its latency, recovery strategies, retrieved memories, Ollama version and model digests, and every host the browser contacted. The analysis reports success, FCR and FRR with Wilson intervals, per-category breakdowns, exact McNemar tests and task-level bootstrap intervals [47] for differences to the full system, and the overhead of Φ relative to baseline (a). A published agent framework as a third baseline would need its own model and prompts and is left to the authors' runs.

C. Hypothesis Evaluation Design

a) Objectives and scenarios: The design tests H1–H5 (Section VI) on the 30 test tasks of the harness, 4 of them unsatisfiable, with three trials per task ($N = 90$ episodes per configuration). Trials run in a fixed order so that memory written in trial 1 is available in trials 2 and 3 (H4). Each hypothesis compares the full system with one configuration that changes only its independent variable (Table III); model, sampling, budget and tasks are held constant.

b) Metrics: With TP the episodes reported complete and achieved under $\Phi^\star$, TN those neither reported nor achieved, and $\mathcal{T}$ all episodes of a configuration:

$$\mathrm{TCR} = \mathrm{TP}/|\mathcal{T}|, \qquad \mathrm{VA} = (\mathrm{TP}+\mathrm{TN})/|\mathcal{T}|, \tag{3}$$

$$\mathrm{ETR} = |\{\tau \in \mathcal{T}_c : \Phi^\star(\tau) \text{ decidable from evidence}\}|/|\mathcal{T}_c|, \tag{4}$$

$$\mathrm{MCS} = \tfrac{1}{|G|}\textstyle\sum_{g\in G} \mathbb{1}[\text{trials of task } g \text{ agree}], \tag{5}$$

$$\mathrm{AER} = |\{\tau : \text{defined terminal status}\}|/|\mathcal{T}|, \tag{6}$$

together with FCR and FRR (Eq. (2)), the median wall-clock time per episode and per verification decision, model calls per episode, and the recovery rate (episodes that reach success after at least one recovery attempt, over episodes with a recovery attempt). ETR is judged by a person who sees only the stored evidence of an episode, blind to its configuration and to $\Phi^\star$; AER counts an episode as reliable if it ends as COMPLETED, FAILED or BLOCKED, and unreliable on a crash or time-out; MCS is over the task set G.

c) Comparison and decision rule: Paired binary outcomes (success, false completion, correct verification) are compared by the exact McNemar test, and differences in rates by a task-level bootstrap 95% interval; a hypothesis is supported only if the interval excludes zero in the expected direction. The analysis script implements these tests (Section VII-B). Two further approaches are part of the design but not implemented: per-step evidence-based verification, which checks each step's expected outcome rather than only the final state, and a combined check that requires both Φ and the self-check to pass.

TABLE IV

COMPLETION GATE ON CONSTRUCTED END STATES. n^-/n^+: STATES THAT DO NOT / DO MEET THE GOAL UNDER THE ORACLE. W: UNSATISFIED STATES WITHHELD (A DISPATCH-ONLY RULE WITHHOLDS NONE); FR: SATISFIED STATES REJECTED; FC/R: FALSE COMPLETIONS OVER STATES REPORTED COMPLETE (FCR). BRACKETS: WILSON 95% INTERVALS. p: EXACT MCNEMAR TEST AGAINST THE DISPATCH-ONLY RULE ON THE SAME STATES. UPPER BLOCK: CURRENT GATE BY CATEGORY; LOWER BLOCK: TOTALS, INCLUDING THE GATE BEFORE THE FIXES. HELD-OUT 2 WAS FROZEN IN A COMMIT BEFORE THE GATE WAS RUN ON IT. SOURCE: `GATE_STATS.CSV`.

	Design (38 states)			Held-out 2 (42 states)		
Category	n^-/n^+	W	FR	n^-/n^+	W	FR
Search	4/3	4	0	3/3	2	0
Navigation	6/2	5	0	4/4	1	0
Bot check	4/2	3	0	4/2	3	1
Text entry	3/3	3	0	3/3	1	0
Extraction	4/3	4	1	5/5	0	0
Multi-step	2/2	1	0	3/3	2	0

Set	Gate	W / n^-	FR / n^+	FC/R	p
Design	before	15/23 [.45,.81]	3/15 [.07,.45]	8/20	<.001
Design	after	20/23 [.68,.95]	1/15 [.01,.30]	3/17	<.001
Held-out 1	before	1/5 [.04,.62]	1/7 [.03,.51]	4/10	1.0
Held-out 1	after	2/5 [.12,.77]	1/7 [.03,.51]	3/9	.50
Held-out 2	after	9/22 [.23,.61]	1/20 [.01,.24]	13/32	.004
Held-out 1+2	after	11/27 [.25,.59]	2/27 [.02,.23]	16/41	.001

VIII. RESULTS

All results concern the browser backend and its components. None measures end-to-end task success.

A. Completion Gate (G1)

Table IV reports the gate study. On the code before the fixes, the pre-check and Φ withheld 15 of the 23 unsatisfied design states and rejected 3 of the 15 satisfied ones; inside the live verification node they withheld only 14 (empty planner reasoning) or 13 (verbose reasoning), because the node let a premature `complete` mark a pending step done and took the planner's reasoning for an answer. After the fixes all three modes agree on every state of every set. On the design states the gate withholds 20 of 23 (exact McNemar $p < 0.001$ against dispatch-only) and corrects 7 states relative to the earlier gate without breaking any ($p = 0.016$); this gain is optimistic by construction, because the fixes were designed on these states.

The held-out sets give the honest estimate. On held-out 2, frozen before testing, the gate withholds 9 of 22 unsatisfied states (0.41; 95% interval 0.23–0.61; $p = 0.004$) and rejects 1 of 20 satisfied ones; pooled with held-out 1, it withholds 11 of 27 (0.25–0.59) and rejects 2 of 27. Its misses follow from the rules. The answer check X is triggered by words such as

TABLE V
ABLATION OF THE GATE ON ALL 92 CONSTRUCTED STATES: FALSE COMPLETIONS (FC) AND FALSE REJECTIONS (FR) WITH ONE CONJUNCT OF Φ, OR THE BOT-CHECK HALT β, REMOVED. p: EXACT MCNEMAR TEST OF CORRECT DECISIONS AGAINST THE FULL GATE ON THE DESIGN STATES (PAIRED BY STATE). ON THE HELD-OUT SETS NO REMOVAL CHANGES MORE THAN THREE DECISIONS. SOURCE: VERIFICATION_GATE_ABLATION.CSV.

	Design (23/15)			Held-out 1 (5/7)		Held-out 2 (22/20)	
Removed	FC	FR	p	FC	FR	FC	FR
none (full Φ)	3	1	–	3	1	13	1
E error, sign-in	8	1	.06	4	1	14	1
H search home	7	1	.12	3	1	15	1
Q query match	4	1	1.0	3	1	13	1
T typed text	6	1	.25	3	1	14	1
X answer	6	0	.62	3	1	13	1
R pending steps	4	1	1.0	3	1	15	1
β bot check	5	1	.50	4	0	15	0

"extract" and "information about", so requests phrased "tell me", "what year", "find" or "get me" are not checked at all, and when X fires it accepts any answer that repeats a goal term ("Tokyo is the capital of Japan" for the population of Tokyo): none of the 5 unsatisfied extraction states is withheld. Text-entry requests phrased "write … in" or "fill … with" are not parsed; error pages titled "410 Gone" or "Something went wrong", a redirect to a different page, a press-and-hold human check, a results page for another query requested as "Google '…' ", and an add-to-cart click that added nothing all pass. The one false rejection is a page that merely mentions reCAPTCHA. Hand-written rules thus fix the cases they were written for and generalize only partially, which answers G1 with a qualified yes: the gate withholds a significant share of premature completions at a low false-rejection rate, but far fewer than on the states it was designed on.

B. Contribution of Each Conjunct (G2)

Table V removes one conjunct at a time. On the design states every removal raises false completions, most for the error check E (3→8) and the search-home check H (3→7), but with 23 unsatisfied states no single removal is significant ($p \geq 0.06$). On the held-out sets each removal changes at most three decisions, and removing Q or X changes none: these conjuncts rarely fire on phrasings they were not written for. Only two removals trade false completions for false rejections: the bot-check halt β (held-out 2: 13→15 FC, 1→0 FR) and, on the design states, the answer check X (3→6 FC, 1→0 FR). Evidence types thus contribute on the states that trigger them, and the gate's coverage, not the strength of any single check, limits it.

C. Overhead

Φ makes no model call and no browser action; it reads the page through the browser protocol. Over the 83 decisions that evaluate Φ the median latency was 26.8 ms (90th percentile 29.5 ms, maximum 34.6 ms; each the median of 20 repetitions), and decisions stopped by the pre-check took at most 13.4 ms. Its added tokens and API cost are zero; the added steps that a rejection causes can only be measured end to end (Section VIII-E).

D. Locality (G3) and Oracle Validity

To test locality we ran 70 full agent episodes (the 10 development tasks under 7 configurations, six of them with memory enabled) through the real control loop and Chromium against the local sites, with a scripted stand-in for the Ollama API on the loopback interface, and logged every socket connection the agent process attempted. All 555 attempts went to 127.0.0.1, and no page request left the loopback interface. The stand-in always claims completion, so these episodes say nothing about task success. The repository's test suite runs this check, together with tests that a remote model address is refused and that Chroma telemetry is off; 144 of its 147 tests pass, and the 3 failures need live websites.

The end-to-end oracle agreed with the intended verdict on all 114 scripted trajectories: no false negative among 37 achieved trajectories (95% interval 0–0.09) and no false positive among 77 unachieved ones (0–0.05). The trajectories were written by the oracle's author, so this bounds the oracle's errors on anticipated behaviors only.

E. End-to-End Evaluation

We have not run the end-to-end evaluation with a language model: the environment used for this paper could not download model weights. Consequently this paper makes no claim about task success, about the gate's effect on success or on false completions of the full agent, about the baselines, or about the contributions of memory, retrieval and the recovery strategies; Table II marks these components as unevaluated. The harness, task set and analysis that produce these measurements are released and run with one command on a machine with Ollama.

IX. DISCUSSION

a) What the evidence supports: A rule over observed state, evaluated inside the agent loop, withholds a significant share of premature completions on constructed browser states at a low false-rejection rate and a cost of tens of milliseconds without model calls. Its defects showed up only when it was tested inside the live node, and its coverage on new phrasings is the binding limit. The evidence does not show that the agent completes tasks.

b) Rules and model judges: The held-out results argue against extending hand-written web rules indefinitely: each rule is keyed to the wording it was written for. Model-based judges [7], [26] cover broader goals but add inference cost and are models judging models [9]. The self-verification baseline in our harness measures exactly this alternative on the same tasks. We expect the two to be complementary—exact postconditions where the environment exposes structured state, a judge as a second opinion where it does not—with both inputs kept as evidence.

X. Safety, Privacy and Autonomy

a) Threat model: The user and the host are trusted; content the agent reads is not. We consider (i) instructions injected through web pages [31]; (ii) high-impact actions without consent, such as posting, sending, purchasing or deleting; (iii) exposure of credentials and of screenshots that show sensitive data; and (iv) disclosure of user data to parties other than the site the user asked the agent to use, in particular to a model provider. A compromised host, browser or model is out of scope.

b) Privacy property: We call the system privacy-preserving in this sense: *model inference is restricted to the user's machine, and task records, memory and evidence are stored only on its disk.* The model client refuses a non-loopback endpoint unless the user sets an explicit override, Chroma telemetry is off, and in 70 agent episodes the agent process opened no non-loopback connection (Section VIII-D). The property does not cover: data the user asks the agent to enter on a website, which reaches that website; model weights and Chroma's default embedding model, which are downloaded once; unencrypted local storage; or traffic of a different configuration. The egress test observed the agent's Python process and the browser's page requests, not background traffic of the Chromium binary.

c) Autonomy and what the implementation provides: The agent acts only in the browser and exposes no file, process or command action. Before navigation, a task whose parsed action is posting, e-mailing, purchasing, deleting or transferring, or whose risk level is high or critical, waits for the user's approval; intent and risk come from the request alone. Bot checks stop the task for the user. A sanitizer replaces 17 instruction-override, delimiter, exfiltration and role-manipulation patterns in element text. The service listens on loopback, and every model call is logged with destination and payload sizes.

d) What it does not provide: The sanitizer is a fixed denylist, which systematic evaluations find insufficient [32]; titles, extracted text and memory are not filtered. The action label and risk level that trigger approval come from a small model and are decided once per task, so an injected instruction that steers the planner to submit a form mid-task is not gated. The API has no authentication, typed text is stored in clear, and screenshots are not redacted. We make no security claim. The browser hides common automation signals; we consider this acceptable only for a personal assistant acting at human pace on its user's requests, and deployments should disable it where a site's terms prohibit automated access.

XI. Limitations and Threats to Validity

a) Internal validity: One author wrote all gate states and labels and designed the fixes on the design states, so the design results are optimistic. Held-out 2 was frozen before testing, but its author knew the gate's rules; independent states and labels would give a less biased estimate. The live mode stands in for the planner with empty or generic reasoning, not model output.

b) External validity: All gate measurements use small, static pages served offline; real pages are larger and change, and latency will grow. No result involves a language model or a live website, so the agent is not shown to complete tasks. The end-to-end harness uses fictional, self-hosted sites, which avoids memorized answers and site drift but is simpler than real sites.

c) Statistical power: The sets are small: with 22 unsatisfied held-out 2 states the 95% interval of the withheld share spans 0.23–0.61, and the ablation cannot separate conjuncts that change one or two decisions.

d) Construct validity: FCR and FRR depend on the oracle. The browser predicate is keyed to request wording, so its coverage depends on how users phrase requests; when no keyword fires, it reduces to an error check.

e) System limitations: Planning with 1.5B–7B models is weak; vision-grounded clicks are not individually checked; two of the four recovery strategies have no distinct behavior; evidence files carry no integrity protection.

XII. Future Work

The first step is to run the released end-to-end protocol with local models, which answers the open questions on task success, the gate's effect on the full agent, the self-verification baseline, memory and recovery. For the gate, we will derive per-step postconditions from the planner's expected outcomes instead of request wording, add a local vision-language judge as a second opinion, and have the held-out states labelled independently. Beyond the browser, the computer backend of Appendix A would extend the gate to desktop applications, files and processes, behind deterministic pre-action permission checks.

XIII. Conclusion

Agentic Pilot is a local browser agent built on the rule that an executed action is not a completed task: it may report completion only when a predicate over the observed page holds, and it keeps the inputs of that decision as evidence. Inside the live loop the predicate withheld most premature completions on the states it was designed on and 41% on states frozen before testing, at a median of 27 ms and no model call; its coverage of new phrasings, not its cost, is the limit. Model traffic is restricted to the user's machine, and the end-to-end harness that will measure task success is released with an oracle validated on scripted trajectories.

Acknowledgment

An AI system (Anthropic's Claude) assisted with analyzing the source code, implementing the experiment harness, running the reported component experiments, searching and checking references, and drafting the text of this paper. The authors reviewed, verified and revised all content and take full responsibility for it.

TABLE VI
COMPLETION POSTCONDITIONS BY ENVIRONMENT AND THEIR STATUS IN THE CODE. IMPL.: IMPLEMENTED AND USED BY THE AGENT; UNIT: IMPLEMENTED AND UNIT-TESTED, NOT CALLED BY THE AGENT; PROP.: PROPOSED DESIGN. h_0: HASH BEFORE THE ACTION; (pid, t_c): PROCESS ID AND CREATION TIME.

Env.	Example postcondition	Status
Browser	no error or sign-in page; results page for the quoted query; requested text present; answer mentions the goal; no pending step	Impl.
Browser	bot-check page detected ⇒ BLOCKED	Impl.
File	path exists and size $\geq n$	Unit
File	created / overwritten: sha256 = sha256(content); moved: source absent and sha256(dst) = h_0(src); trashed: path absent and trash entry present	Prop.
Process	launched: (pid, t_c) alive; terminated: (pid, t_c) gone within a timeout	Prop.
Window	foreground title matches; window with title exists	Prop.
Desktop UI	accessibility value of a control equals the input; control state contains *checked*	Prop.
Read-only	workspace manifest (paths, sizes, mtimes) unchanged	Prop.

TABLE VII
END-TO-END EVALUATION DESIGN (IMPLEMENTED IN BENCHMARKS/E2E, NOT YET RUN WITH A MODEL). EACH CONFIGURATION CHANGES ONE FACTOR OF THE FULL SYSTEM; ALL USE THE SAME TASKS, MODEL, SAMPLING AND BUDGET.

Configuration	Change	Question
`full`	–	task success, FCR, FRR, cost
`no_verification`	every completion claim accepted	baseline (a); gain and overhead of Φ
`self_verification`	planner model judges its own completion	baseline (b)
`gate_without_`c	conjunct c removed	per-evidence ablation
`no_memory`	memory and RAG off	contribution of memory
`no_evidence`	no per-action execution records	traceability (H3)
`no_recovery`	failures not retried	contribution of recovery
`recovery_`s`_only`	strategy s pinned	per-strategy recovery
`single_model`, `vision_only`	routing off; no element list	routing, grounding

APPENDIX A
COMPUTER BACKEND (DESIGN, NOT IMPLEMENTED)

The repository contains a desktop executor that wraps PyAutoGUI and psutil, a text-file helper and a file-existence predicate. None is connected to the control loop: no node, action type or API route calls them, and the file helper does not confine paths. The rest of this appendix is a design.

a) Observation and grounding: The backend would observe the foreground window through its accessibility tree (Windows UI Automation, macOS Accessibility, AT-SPI) into a manifest shaped like the DOM manifest, falling back to a vision model on a window screenshot. File and process actions are grounded in typed state (paths, hashes, process identity).

b) Postconditions and permission tiers: Each mutating action carries exact postconditions (Table VI) captured with before/after values. Because many local actions cannot be undone, a postcondition checked afterwards is no protection on its own, so an `action_gate` node between planning and execution would compute a tier from the concrete action: T0 (read-only) runs freely; T1 (reversible, with a recorded inverse) runs automatically only if the user enabled it; T2 (destructive) and T3 (network-capable programs, credentials) always need approval bound to a hash of the exact action. Paths outside allow-listed roots, inline interpreters and privilege elevation would be refused. A design in which untrusted content cannot change the control flow [37] is a stronger alternative.

APPENDIX B
END-TO-END TASK SET

The 40 tasks (`benchmarks/e2e/tasks.json`) are distributed as follows (development/test): extraction 2/8, form submission 2/7, multi-step 1/5, navigation 2/2, search 1/2, text entry 1/2, unsatisfiable 1/4; tiers 1–4 contain 3/6, 4/11, 3/12 and 0/1 tasks. The test split uses all five sites; the notes site appears only there. The harness (`run_e2e.py`) takes the split, the number of trials and a list of the configurations of Table VII; the analysis script (`analyze_e2e.py`) writes result tables with Wilson intervals, McNemar tests and bootstrap intervals.

APPENDIX C
ILLUSTRATIVE EVALUATION AND DISCUSSION

Disclaimer. The numerical values presented in this section are hypothetical and illustrative, intended solely to demonstrate the proposed evaluation framework. They are not obtained from actual experimental execution or empirical measurements. They are not results of this paper, and no statistical test was applied to them.

This appendix shows how the design of Section VII-C would report evidence on H1–H5. The values were chosen by hand to be plausible and internally consistent: every row of Table VIII sums to $N = 90$ episodes (78 on satisfiable and 12 on unsatisfiable tasks), and every rate follows from its counts by Eqs. (2)–(3). Approaches 4 and 5 and the unstructured baseline of Table IX are not implemented.

a) Illustrative interpretation (not findings): If measured values resembled Table VIII, H1 would be supported: the rule-based check would lower FCR from .474 to .154 while TCR stays similar, because a rejected claim returns control to the planner rather than ending the episode. H2 would be supported on accuracy (.867 against .678) and latency (27 ms against a model call), and the self-check's false rejections would show the cost of a model judging from truncated page text. Approach 5 would trade a lower FCR for a higher FRR, the trade-off already seen when β is removed (Section VIII-B). In Table IX, H3 would be supported if the auditor resolves more episodes with execution records; H4 would rest on a small difference on repeated trials that $N = 60$ episodes could not separate from noise, and the episodes misled by memory would need inspection; H5 could be tested only after the unstructured baseline is built.

TABLE VIII

HYPOTHETICAL VALUES FOR H1 AND H2: COMPLETION DECISIONS OF FIVE VERIFICATION APPROACHES OVER $N = 90$ EPISODES. TP: REPORTED AND ACHIEVED; FC: REPORTED, NOT ACHIEVED; FR: ACHIEVED, NOT REPORTED; TN: NEITHER. *NOT IMPLEMENTED. THE DECISION TIME OF APPROACH 3 IS SET TO THE COMPONENT LATENCY MEASURED IN SECTION VIII-C; ALL OTHER VALUES ARE NOT MEASURED.

Approach	TP	FC	FR	TN	TCR	FCR	FRR	VA
1 No verification	41	37	0	12	.456	.474	.000	.589
2 LLM self-verification	40	24	5	21	.444	.375	.111	.678
3 Rule-based (Φ)	44	8	4	34	.489	.154	.083	.867
4 Per-step evidence*	45	5	5	35	.500	.100	.100	.889
5 Combined (3 and 2)*	44	3	7	36	.489	.064	.137	.889

Approach	1	2	3	4*	5*
Median episode time (s)	48	61	55	58	67
Median decision time (ms)	0	1,900	27	35	1,930
Model calls per episode	6.1	8.4	7.2	7.5	9.0

TABLE IX

HYPOTHETICAL VALUES FOR H3–H5, FULL SYSTEM AGAINST ONE CHANGED FACTOR. *NOT IMPLEMENTED. NOT MEASURED.

H	Metric	Full system	Changed factor
H3	ETR (reported completions)	49/52 (.94)	42/52 (.81)
	median audit time per episode	1.6 min	2.9 min
H4	TCR, trial 1	14/30 (.47)	14/30 (.47)
	TCR, trials 2–3	30/60 (.50)	29/60 (.48)
	MCS	22/30 (.73)	20/30 (.67)
	actions per success	6.2	6.6
	episodes misled by memory	3	–
H5	AER	88/90 (.98)	76/90 (.84)*
	TCR	44/90 (.49)	35/90 (.39)*
	recovery rate	11/29 (.38)	–*

Changed factor: H3 `no_evidence`; H4 `no_memory`; H5 unstructured loop*.

b) Possible outcomes and failure scenarios: The measured data may differ in ways the design is meant to reveal. The rule-based check may gain little end to end if the planner rarely claims completion prematurely, or if requests use wordings that trigger no conjunct (Section VIII-A); its FCR would then approach that of approach 1. The self-check may outperform the rules on extraction tasks, where the rules missed every unsatisfied held-out state. Memory may lower success if failed outcomes are retrieved as guidance. Episode time-outs on CPU inference would lower AER for every configuration and mask differences between them.

c) Requirements for validation: Each hypothesis is supported only by measured data from the protocol of Sections VII-B and VII-C, with the paired tests stated there; these illustrative values must then be replaced, not combined with measurements. H3 additionally needs a blind human auditor, and H5 an implemented unstructured baseline. With 90 episodes per configuration, differences of a few percentage points, as in H4 above, are below the resolution of the design.

REFERENCES

[1] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[2] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, "WebVoyager: Building an end-to-end web agent with large multimodal models," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL) (Volume 1: Long Papers)*, 2024, pp. 6864–6890.

[3] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu *et al.*, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024.

[4] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "GPT-4V(ision) is a generalist web agent, if grounded," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 61 349–61 385.

[5] C. Zhang, L. Li, S. He, X. Zhang, B. Qiao, S. Qin, M. Ma, Y. Kang, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang, "UFO: A UI-focused agent for Windows OS interaction," in *Proc. 2025 Conf. Nations of the Americas Chapter Assoc. Comput. Linguistics: Human Language Technologies (NAACL-HLT) (Volume 1: Long Papers)*, 2025, pp. 597–622.

[6] N. Lukas, A. Salem, R. Sim, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Analyzing leakage of personally identifiable information in language models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2023, pp. 346–363.

[7] J. Pan, Y. Zhang, N. Tomlin, Y. Zhou, S. Levine, and A. Suhr, "Autonomous evaluation and refinement of digital agents," in *Proc. Conf. Language Modeling (COLM)*, 2024.

[8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[9] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837.

[11] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.

[12] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.

[13] G. Kim, P. Baldi, and S. McAleer, "Language models can solve computer tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.

[14] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Trans. Mach. Learn. Res. (TMLR)*, 2024.

[15] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2Web: Towards a generalist agent for the web," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.

[16] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, "Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V," arXiv:2310.11441, 2023.

[17] J. Y. Koh, R. Lo, L. Jang, V. Duvvur, M. C. Lim, P.-Y. Huang, G. Neubig, S. Zhou, R. Salakhutdinov, and D. Fried, "VisualWebArena: Evaluating multimodal agents on realistic visual web tasks," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024, pp. 881–905.

[18] A. Drouin, M. Gasse, M. Caccia, I. H. Laradji, M. Del Verme, T. Marty, D. Vazquez, N. Chapados, and A. Lacoste, "WorkArena: How capable are web agents at solving common knowledge work tasks?" in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 11 642–11 662.

[19] R. Bonatti, D. Zhao, F. Bonacci *et al.*, "Windows agent arena: Evaluating multi-modal OS agents at scale," arXiv:2409.08264, 2024.

[20] S. Agashe, J. Han, S. Gan, J. Yang, A. Li, and X. E. Wang, "Agent S: An open agentic framework that uses computers like a human," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[21] C. Zhang, Z. Yang, J. Liu, Y. Li, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, "AppAgent: Multimodal agents as smartphone users," in *Proc. 2025 CHI Conf. Human Factors Comput. Syst. (CHI)*, 2025, pp. 70:1–70:20.

[22] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Dong, M. Ding, J. Tang *et al.*, "CogAgent: A visual language model for GUI agents," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 14 281–14 290.

[23] K. Cheng, Q. Sun, Y. Chu, F. Xu, L. YanTao, J. Zhang, and Z. Wu, "SeeClick: Harnessing GUI grounding for advanced visual GUI agents," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL) (Volume 1: Long Papers)*, 2024, pp. 9313–9332.

[24] Y. Qin, Y. Ye, J. Fang *et al.*, "UI-TARS: Pioneering automated GUI interaction with native agents," arXiv:2501.12326, 2025.

[25] Y. Lu, J. Yang, Y. Shen, and A. Awadallah, "OmniParser for pure vision based GUI agent," arXiv:2408.00203, 2024.

[26] M. Sumyk and O. Kosovan, ""are we done yet?": A vision-based judge for autonomous task completion of computer use agents," arXiv:2511.20067, 2025.

[27] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[28] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods Natural Lang. Process. and 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[29] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST)*, 2023.

[30] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023.

[31] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec)*, 2023.

[32] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Secur. Symp. (USENIX Security)*, 2024, pp. 1831–1847.

[33] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings Assoc. Comput. Linguistics: ACL 2024*, 2024, pp. 10 471–10 506.

[34] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024.

[35] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[36] T. Kuntz, A. Duzan, H. Zhao, F. Croce, Z. Kolter, N. Flammarion, and M. Andriushchenko, "OS-Harm: A benchmark for measuring safety of computer use agents," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, 2025.

[37] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," arXiv:2503.18813, 2025.

[38] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu *et al.*, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[39] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai *et al.*, "Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution," arXiv:2409.12191, 2024.

[40] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.

[41] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, p. 186345, 2024.

[42] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong *et al.*, "The rise and potential of large language model based agents: A survey," *Sci. China Inf. Sci.*, vol. 68, p. 121101, 2025.

[43] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[44] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[45] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[46] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[47] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.